

Lezione — Modulo 3C: Gestione dei Processi

Corso: Lab Amministrazione di Sistemi T **Materiale:** `shell_processi_teoriaspansione.pdf` (slide 23–32), `-- LAB -- Gestione di processi [17 marzo] _ Virtuale.pdf` **Prerequisiti:** Modulo 1A/1B (bash scripting), Modulo 3A (systemd e servizi)

Obiettivo

Al termine di questa lezione Lorenzo deve saper lanciare processi in background, monitorarne lo stato con `ps`, `jobs` e `watch`, inviargli segnali con `kill`, sincronizzare processi paralleli con `wait`, e usare `nohup/disown` per disancorare un processo dalla shell.

Concetti Chiave

1. Processo: cos'è e come si identifica

Ogni comando lanciato dalla shell diventa un **processo**. I processi sono identificati in due modi:

- **PID** (Process ID): numero univoco globale nel sistema, assegnato dal kernel.
- **Job ID**: numero locale alla shell corrente, usato solo internamente alla sessione. Si riferisce col prefisso %, es. %1.

Un processo agisce **a nome dell'utente che lo ha lanciato** — i permessi del processo sono quelli dell'utente. I processi root possono assumere l'identità di altri utenti (declassandosi, senza possibilità di tornare indietro).

Come nasce un processo: la shell chiama `fork()` (copia del processo corrente) poi `exec()` (sostituisce il codice con il programma da eseguire). Ogni processo figlio eredita i file descriptor aperti dal padre.

2. Processi in foreground e background

Per default un comando lanciato dalla shell è in **foreground**: la shell aspetta che termini prima di mostrare il prompt.

Per lanciarlo in **background** si aggiunge `&` alla fine della riga:

```
sleep 1000 &
```

La shell risponde con due valori: - Il **job id** tra parentesi quadre: `[1]` - Il **PID** del processo: `12345`

Variabile speciale \$!: contiene sempre il PID dell'ultimo processo lanciato in background. Da usare subito dopo il `&` per catturarlo:

```
sleep 1000 &
A=$!
echo $A          # stampa il PID
```

Un processo in background: - Non riceve più input dal terminale - Continua però a scrivere su STDOUT e STDERR (output appare a schermo a meno di redirect)

Se hai lanciato un comando in foreground e vuoi metterlo in background: 1. Ctrl+Z → invia SIGTSTP → il processo si **sospende** (stato stopped) 2. bg %job_id → invia SIGCONT e rimette il job in esecuzione in background

3. Segnali

I segnali sono **eventi asincroni** notificati dal kernel a un processo. Sono il meccanismo principale per controllare i processi dall'esterno.

Cosa può fare un processo alla ricezione di un segnale (signal disposition): - Terminare (con o senza core dump) - Ignorarlo - Sospendersi (stato stop) - Riprendersi dallo stop (cont)

Ogni processo può registrare un **handler** personalizzato per gestire un segnale, tranne per SIGKILL e SIGSTOP — questi non possono essere bloccati, ignorati o intercettati.

Segnali importanti:

Segnale	Numero	Scorciatoia tastiera	Effetto default
SIGTERM	15	—	Termina il processo (graceful)
SIGKILL	9	—	Termina il processo (forza, non intercettabile)
SIGINT	2	Ctrl+C	Interrompe il processo
SIGQUIT	3	Ctrl+\	Termina con core dump
SIGTSTP	20	Ctrl+Z	Sospende il processo
SIGSTOP	19	—	Sospende (non intercettabile)
SIGCONT	18	—	Riprende un processo sospeso
SIGHUP	1	—	Hangup (chiusura terminale)

Invio segnali con kill:

```
kill <PID>           # invia SIGTERM (default)
kill -9 <PID>         # invia SIGKILL
kill -STOP <PID>      # sospende il processo
kill -CONT <PID>      # riprende il processo
kill -l               # elenca tutti i segnali supportati
kill -<N> <PID>       # invia il segnale numero N
```

Con PID negativi si invia il segnale all'intero **process group**: kill -9 -<PGID>

trap — handler in bash:

trap permette di definire cosa fare quando la shell riceve un segnale:

```
trap "echo 'Segnale ricevuto'" SIGTERM
trap "rm -f /tmp/miofile" EXIT    # eseguito all'uscita dello script
```

Pseudo-segnali bash (per debug): - DEBUG — eseguito prima di ogni comando - ERR — eseguito quando un comando fallisce - EXIT — eseguito all'uscita dello script - RETURN — eseguito al ritorno da una funzione o dopo source

Nota critica: i signal handler NON vengono ereditati dai processi figli. Quando bash esegue un comando esterno, il processo bash non è schedulato fino alla terminazione del figlio — i segnali arrivano ma vengono processati solo dopo.

4. Comandi di monitoraggio

ps — snapshot dei processi in esecuzione:

```
ps                # processi della shell corrente
ps $PID           # processo specifico (vuoto se non esiste)
ps aux            # tutti i processi del sistema
ps hp $PID        # output senza header, utile negli script
```

Le colonne di **ps**: PID, TTY, STAT (stato), TIME (CPU usato), CMD.

Codici di stato (STAT): S = sleeping, R = running, T = stopped, Z = zombie.

jobs — lista dei job della shell corrente:

```
jobs              # mostra job id, stato, comando
jobs -l           # aggiunge il PID
```

fg — riporta un job in foreground:

```
fg %1             # riporta il job 1 in foreground
```

watch — esegue un comando periodicamente e mostra l'output aggiornato:

```
watch "ps aux | grep sleep | grep -v grep"
```

Di default aggiorna ogni 2 secondi. Utile per osservare un processo che cambia stato nel tempo.

5. Sincronizzazione: wait

wait blocca l'esecuzione dello script fino al completamento dei job in background.

```
sleep 30 &
sleep 40 &
wait          # attende entrambi
echo "Tutti terminati"
```

Si può aspettare un job specifico:

```
wait $PID      # aspetta solo quel PID
```

Interazione con trap: se durante l'attesa arriva un segnale con handler definito via **trap**, **wait** esce immediatamente con exit code > 128, l'handler viene eseguito, e l'esecuzione riprende dopo il **wait**.

6. Modificatori per processi in background

nohup — immunità all'hangup:

Normalmente quando si chiude la shell, il kernel invia **SIGHUP** a tutti i job in background, terminandoli. **nohup** evita questo:

```
nohup sleep 1000 &           # sopravvive alla chiusura della shell
```

nohup reindirige anche lo stdout su **nohup.out** se non diversamente specificato.

disown — rimuove un job dalla job table della shell:

```
sleep 1000 & disown          # lancia e rimuove subito dalla job table
disown %1                    # rimuove il job 1 già in background
disown -h %1                 # implementa anche l'immunità a SIGHUP
```

Differenza con **nohup**: **disown** agisce su processi già avviati; **nohup** si usa all'avvio.

nice — priorità di scheduling:

```
nice -n 10 comando           # lancia con niceness +10 (priorità più bassa)
nice nohup long_calc &       # combinazione comune
```

La **niceness** va da -20 (massima priorità) a +19 (minima). Valori negativi solo da root. Default di **nice** senza argomenti: +10.

7. Array bash per tracciare processi

In bash gli array possono avere **indici non consecutivi**. Utile per associare un comando al suo PID:

```
COMANDO1="sleep 10"
COMANDO2="sleep 20"

$COMANDO1 & PID[$!]=$COMANDO1
$COMANDO2 & PID[$!]=$COMANDO2

echo ${!PID[@]}           # stampa gli indici (= i PID)
echo ${PID[@]}            # stampa i valori (= i comandi)
echo ${PID[1234]}         # valore associato al PID 1234
```

8. File temporanei con mktemp

Quando più processi devono scrivere output separati (non possono condividere lo stesso file senza sincronizzazione), si usa **mktemp** per creare file temporanei sicuri:

```
FA=$(mktemp)              # crea /tmp/tmp.XXXXXX e restituisce il path
FB=$(mktemp)
# ... usare $FA e $FB come destinazioni separate ...
rm -f "$FA" "$FB"         # pulizia obbligatoria
```

mktemp risolve il problema della race condition (due processi che creano contemporaneamente un file con lo stesso nome).

Comandi di Riferimento

Comando	Sintassi	Descrizione
&	comando &	Lancia in background
#!	echo #!	PID dell'ultimo processo in background
jobs	jobs [-l]	Lista job della shell corrente
fg	fg %job_id	Riporta in foreground
bg	bg %job_id	Manda/riavvia in background
kill	kill [-segnale] PID	Invia segnale a processo
ps	ps [aux] [PID]	Mostra processi
wait	wait [PID]	Aspetta terminazione job
watch	watch "comando"	Esegue comando periodicamente
nohup	nohup cmd &	Lancia immune a SIGHUP
disown	disown [%job]	Rimuove job dalla job table
nice	nice [-n N] cmd	Lancia con priorità modificata
trap	trap "codice" SEGNALE	Definisce handler per segnale
mktemp	FA=\$(mktemp)	Crea file temporaneo sicuro

Esercizi Guidati

Esercizio 1 — Processi in background e segnali

Avvia la VM e segui la sequenza commentando l'output dopo ogni comando:

```
# 1. Lancia sleep in background e cattura il PID
sleep 1000 &
A=$!
echo $A          # stampa il PID + numero tipo 1234

# 2. Verifica che il processo esista
ps $A            # mostra una riga con PID e stato "S" (sleeping)
jobs             # mostra [1]+ Running sleep 1000 &
jobs -l          # come jobs ma con il PID esplicito

# 3. Sospendi il processo
kill -STOP $A
ps $A            # stato diventa "T" (stopped)
jobs             # mostra [1]+ Stopped sleep 1000

# 4. Riprendi il processo
kill -CONT $A
ps $A            # torna "S"
```

```
jobs                                # torna "Running"

# 5. Termina il processo
kill $A                            # invia SIGTERM
ps $A                             # riga vuota: processo non esiste più
jobs                               # mostra [1]+ Terminated sleep 1000
```

Esercizio aggiuntivo — Scrivi uno script che lancia `sleep 10` e `sleep 20` in parallelo e ogni 5 secondi stampa il loro stato (in esecuzione / terminato). Prima di leggere la soluzione, prova da solo.

Esercizio 2 — nohup e disown

```
# Lancia tre sleep in modi diversi, poi chiudi e riapri la shell
# (o usa exit e vagrant ssh), e verifica con ps aux | grep sleep quali sopravvivono

sleep 1000 &                       # legato alla shell → muore con SIGHUP
sleep 1000 & disown                 # rimosso dalla job table → sopravvive
nohup sleep 1000 &                 # immune a SIGHUP → sopravvive
```

Per verificare dopo il logout: `ps aux | grep sleep | grep -v grep`

Esercizio 3 — wait e watch (due terminali)

Apri due sessioni SSH sulla VM (`vagrant ssh` in due terminali distinti).

Terminale 1:

```
sleep 30 &
sleep 40 &
wait
echo "Tutti terminati"
```

Terminale 2 (contemporaneamente):

```
watch "ps auxw | grep sleep | grep -v grep"
```

Osserva come nel Terminale 2 i processi spariscono uno per uno, e come nel Terminale 1 il prompt riappare solo dopo che entrambi sono terminati.

Esercizio 4 — Array con PID

```
COMANDO1="sleep 10"
COMANDO2="sleep 20"

$COMANDO1 & PID[$!]=$COMANDO1
$COMANDO2 & PID[$!]=$COMANDO2
```

```
echo ${!PID[@]}      # dovrebbe stampare due numeri (i PID)
echo ${PID[@]}       # dovrebbe stampare "sleep 10 sleep 20"

# Estrai un singolo elemento (sostituisci XXXXX con uno dei PID stampati sopra)
echo ${PID[XXXXX]}
```

Esercizio 5 — Comandi paralleli sincronizzati

Consegna: scrivi uno script che accetta due directory come argomenti, conta il numero di file in ciascuna **in parallelo** (un processo per directory), aspetta la fine di entrambi i conteggi, e stampa quale directory contiene più file.

Requisiti: - Controllo che i due parametri siano directory valide - I conteggi devono girare in parallelo con `&` - Usare `wait` per sincronizzare - Usare `mktemp` per i file di output temporanei - Fare pulizia dei file temporanei alla fine

Prima di leggere la soluzione del professor Prandini, implementa il tuo script. Poi confronta: la soluzione usa funzioni `testd()` e `conta()` per separare le responsabilità.

Connessioni

Con il modulo precedente (3A — Systemd): - Systemd è esattamente un gestore di processi avanzato: quello che facciamo a mano con `kill`, `wait` e segnali, systemd lo fa in modo strutturato per i servizi di sistema. - Il concetto di “stato di un processo” (running, stopped) è lo stesso di `systemctl status`.

Con Security (S1, S5): - L’enumerazione di processi (`ps aux`) è uno dei primi passi del reconnaissance su un host compromesso — si cerca quali servizi girano e sotto quale utente. - `kill -9` su un processo critico è una forma di denial of service locale. - `nohup cmd &` è il pattern usato per installare backdoor persistenti che sopravvivono al logout.

Riepilogo

- **\$! e &:** il pattern fondamentale per la parallelizzazione in bash — lancia in background, cattura il PID, poi usa `wait` per sincronizzare.
- **Segnali:** il meccanismo kernel per comunicare con un processo; `kill` non “uccide” sempre — invia qualunque segnale; `SIGKILL` e `SIGSTOP` sono gli unici non intercettabili.
- **nohup vs disown:** entrambi svincolano il processo dalla shell, ma `nohup` si usa all’avvio, `disown` su processi già in background; `mktemp` risolve le race condition su file temporanei condivisi.

Collegati

- [[Appunti_modulo3C]]
- [[appunti_modulo3C_gestione_processi]]

Hub: [[master_map_studio]] · [[glossario_sysadm]] · [[concept_maps]] · [[troubleshooting_vm]]
· [[metodo_studio_esami_pratici]]